



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

NIS3364 课程报告

NetworkSniffer: 简单网络嗅探器项目实施报告

学院 电子信息与电气工程学院

专业 信息安全

学号 522031910522

姓名 钟泓逸

2024 年 12 月 17 日

目录

1	项目信息	3
1.1	概述	3
1.2	基本信息	3
1.3	项目文件列表	3
2	功能及函数说明	3
2.1	可视化	3
2.2	网卡侦听功能	5
2.3	过滤侦听	5
2.4	报文分析	6
2.4.1	报文重组功能	6
2.5	记录保存	7
3	运行与测试	7
3.1	运行方式	7
3.2	网卡侦听测试	8
3.2.1	http/https 报文测试	8
3.2.2	报文分析测试	8
3.3	报文重组测试	9
3.4	筛选功能	10
3.5	保存功能	10
4	写在最后	11
4.1	发布地址	11
4.1.1	遇到的问题与解决办法	11
4.1.2	体会与建议	12
A	与大模型交互过程	13
B	报文重组测试代码	14

1 项目信息

1.1 概述

本项目使用 python 语言实现，使用 tkinter 库构建可视化界面，同时使用 scapy 库完成网络包的嗅探、分析与重组功能。

1.2 基本信息

- 开发环境：Windows
- 运行环境：Windows
- 开发语言：python
- 构建工具：pyinstaller

1.3 项目文件列表

- /src: 源代码文件夹，包括：
 - filter.py: 实现过滤器的主要文件
 - packet_processor.py: packet 处理文件
 - sniffer.py: 主文件，实现 sniffer 窗口生成和交互功能
 - storage.py: 用于管理处理后 packet 的储存
 - img (文件夹)
- /dist: 可执行文件文件夹，包括：
 - sniffer.exe
- /test: 测试文件夹，包括：
 - test.py
- requirements.txt: 项目依赖文件

2 功能及函数说明

该项目主要由 tkinter 库实现可视化，实现了网卡侦听、过滤侦听、报文分析、报文重组和结果保存功能。其详细描述与具体功能实现如下所示：

2.1 可视化

项目可视化由 tkinter 实现，其具体实现类为 SnifferApp，包括主界面、报文重组界面、筛选器界面。

报文主界面主要由 TreeView 组件、ComboBox 组件、Textview 组件组成。

- ComboBox 组件用于选择侦听网卡

- TreeView 组件用于显示捕获报文的基本信息，包括序号、时间、源地址、目标地址、协议、长度、详细信息等，以表格方式呈现
- Textview 组件用于显示具体报文的分析信息，包括各层协议的解析以及内层数据的不同编码方式显示

主页面 GUI 如图1所示。

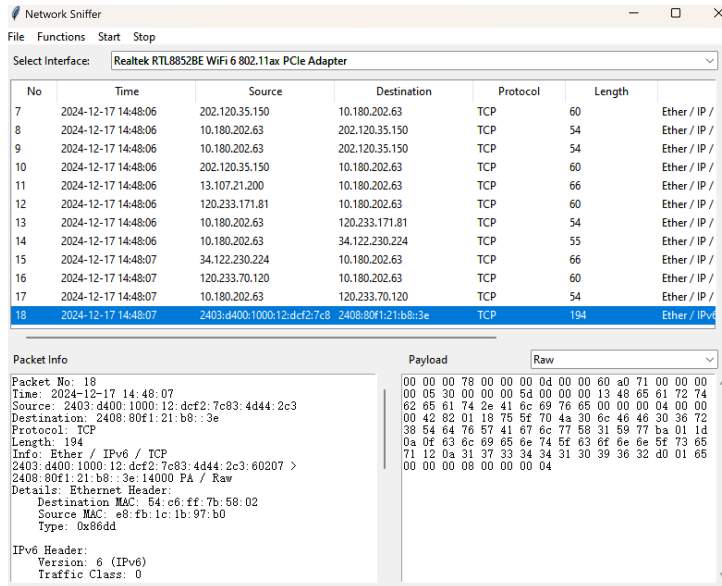


图 1: 主界面 GUI

报文重组界面主要由 Textview 组件、ComboBox 组件、Botton 组件组成，用于用指定编码方式显示选定报文重组后的数据流，并提供保存功能。如图2所示。



图 2: 报文重组界面 GUI

筛选器界面主要由 OptionMenu 组件、Entry 组件组成，用于输入网卡侦听的筛选条件，包括协议、源 IP、目标 IP、源端口、目标端口、源 MAC、目标 MAC 和子网掩码，如图3所示

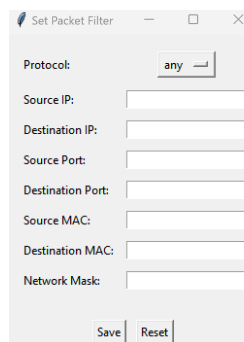


图 3: 筛选器界面

2.2 网卡侦听功能

在主界面的复选框中，程序会检测相关网卡，供用户选择。用户选择网卡后，点击菜单栏中的 `start` 和 `stop` 按钮，可以实现特定网卡的开始侦听/结束侦听。主要实现

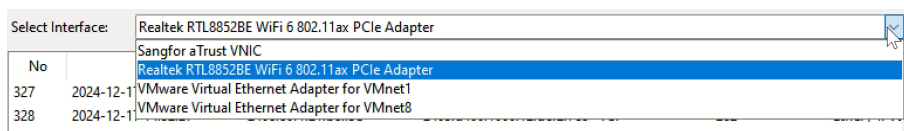


图 4: 特定网卡选择

函数如下所示：

- **get_network_interfaces**: 网卡选择函数
 - 返回参数：字典（网卡信息）
 - 功能：该函数通 WMI 查询获取计算机上启用了 IP 的网络适配器信息（包括 MAC 地址），并且使用 `netifaces` 获取当前计算机上的所有网络接口，获取每个接口的 MAC 地址，通过比对，找到正确接口，并返回网卡信息。
- **run_sniffer/start_sniffing**: 侦听函数
 - 返回参数：无
 - 功能：根据用户选择的网络接口，启动一个新的线程，使用 `scapy` 的 `sniff` 方法在指定接口上捕获数据包，并通过 `enqueue_packet` 方法处理每个捕获的数据包
- **stop_sniffing**: 停止侦听函数
 - 功能：通过 `sniffing` 参数控制停止侦听。
- **extract_packet_info**: 从原始包中提取基础信息并转为 `Packet` 类

2.3 过滤侦听

点击菜单栏的 `Functions - Set Filter`，可以唤起过滤器界面（图3）。过滤器界面一共可以对 8 个字段进行筛选，包括协议、源 IP、目标 IP、源端口、目标端口、源 MAC、

目标 MAC 和子网掩码。其中协议字段为单选，选项内容包括 tcp/udp/icmp/any。过滤器主要由 filter 文件实现，其函数如下所示：

- **filter**：创建过滤器菜单
 - 功能：在用户点击 Save 时，获取各个过滤字段数值并且生成 bpf，传递给 sniff 函数用于过滤。
- **generate_bpf**：bpf 生成函数
 - 功能：根据用户输入的过滤条件生成一个 BPF 过滤器字符串，构建过滤条件并将它们用“and”连接。如果没有过滤条件，默认捕获所有数据包。最后，将生成的 BPF 字符串通过消息框显示并保存。

2.4 报文分析

当选中报文列表中的某一个报文时，会对该报文的具体内容进行分析，逐层解析头部信息，并打印在 Packet Info 框中，如图左5所示。若有对应传输数据，则会将数据以选定编码格式显示在 payload 框中，如图右5所示。其具体函数如下所示：

Packet Info	Payload	Raw
Packet No: 336 Time: 2024-12-17 14:52:30 Source: 10.180.202.63 Destination: 192.168.1.1 Protocol: TCP Length: 1034 Info: Ether / IP / TCP 10.180.202.63:54321 > 192.168.1.1:12345 S / Raw Details: Ethernet Header: Destination MAC: 54:c6:ff:7b:58:02 Source MAC: e8:fb:1c:1b:97:b0 Type: 0x800 IP Header: Version: 4 (IPv4) Header Length: 20 bytes Total Length: 1020 bytes	89 50 4e 47 0d 0a 1a 0a 00 00 00 0d 49 48 44 52 00 00 00 c8 00 00 00 c8 08 06 00 00 00 ad 58 ae 9e 00 00 00 01 73 52 47 42 00 ae ce 1c e9 00 00 0a 12 49 44 41 54 78 5e ed 9d ed 71 1c 37 0c 86 d7 9d 24 95 24 ae c4 4e 25 4a 2a 71 54 89 9d 4a 92 54 92 08 23 ad 75 b2 ee 6e 49 2e 08 80 c0 b3 33 1e ff 10 09 1e 5f e0 99 77 3f 70 7b 1f 36 0e 14 40 81 9b 0a 7c 40 1b 14 40 81 db 0a 00 08 d5 81 02 77 14 00 10 ca 03 05 00 84 1a 40 81 31 05 70 90 31 dd 98 55 44 01 00 29 92 68 b6 39 a6 00 80 8c e9 c6 ac 22 0a 00 48 91 44 b3 cd 31 05 00 64 4c 37 66 15 51 20 0a 20 bf 6f db f6 cb b6 6d 3f bd fc fb 67 db 36 f9 f7 d7 53 1e e4 6f 1c 28 e0 a2 80 37 20 bf 3e 81 f1 f5 60 e7 02 ca 23 a0 b8 d4 47 f9 45 3d 01 11 67 78 e8 c8 80 80 f2 db b6 6d df 3a e6 30 14 05 4e 29 e0 05 48 2f 1c fb 26 71 93 53 e9 66 72 af 02 1e 80 b4 9c 56 1d ed	

图 5: 报文分析栏

- **on_tree_select**：列表选择触发函数
 - 功能：当用户选择报文后，调用分析函数，并将分析内容更新到对应的 UI 中。
- **process_packet**：报文处理函数
 - 功能：获取格式化后的报文详细信息，并且更新对应的 payload 信息
- **format_packet**：详细信息获取函数
 - 功能：该函数获取嗅探到的报文，并且调用 scapy 库的相关功能分析获取对应 ARP/IP/IPv6/IGMP/TCP/UDP/ICMP 层信息并进行格式化处理。
- **update_hex_text** 等：使用对应编码格式更新 payload

2.4.1 报文重组功能

选择列表中的任意报文后，点击菜单 Functions - Track Stream，可以进行报文重组。

- 若选择报文非分片（即不存在 MF 标志），则会弹出提示框，如图6所示。

- 若选择报文有 MF 标志，则会弹出对应重组窗口，可以选择编码格式进行查看并保存，如图7所示。

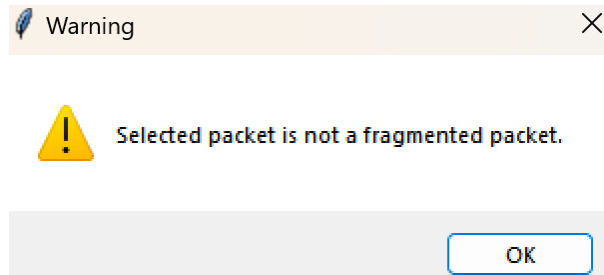


图 6: 标志位为 DF 的报文尝试重组结果

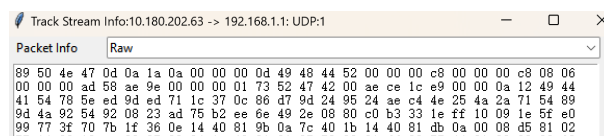


图 7: 标志位为 MF 的报文尝试重组结果

其主要函数如下图所示：

- **track_stream**：完成报文重组的基本逻辑并实现 ui 界面
 - 功能：首先检查报文的标志位查看是否为分片报文，若具有 MF 标志位，则以 (src,dst,proto,id) 为依据，筛选统一数据流报文并排序，组合成完整报文。最后，或许相应信息实现 ui。

2.5 记录保存

一共提供两个报文功能：

- 嗅探到的报文列表基础信息保存
- 数据流重组后的数据信息保存

其主要函数有两个，分别为：

- **save_binary_file**：调用系统文件窗口保存二进制流。
- **save_csv**：调用系统文件窗口保存相应记录至 csv。

3 运行与测试

3.1 运行方式

程序一共有两个运行方式：

- 满足文件 requirements.txt 中的依赖要求后，在 src 文件夹下运行指令运行主程序：

```
1 python sniffer.py
```

- 直接运行 dist 文件夹下的可执行文件 `sniffer.exe`

3.2 网卡侦听测试

3.2.1 http/https 报文测试

使用好大学在线（180.76.151.202）和百度，获取部分对应 http 和 https 报文如下

182	2024-12-17 22:30:10	180.76.151.202	10.180.202.63	TCP	362	Ether / IP / TCP 180.76.151.202:htp > 10.180.202.63:60222 PA / Raw
183	2024-12-17 22:30:10	10.180.202.63	180.76.151.202	TCP	729	Ether / IP / TCP 10.180.202.63:60222 > 180.76.151.202:htp PA / Raw
184	2024-12-17 22:30:10	180.76.151.202	10.180.202.63	TCP	60	Ether / IP / TCP 180.76.151.202:htp > 10.180.202.63:60222 A / Padding

Packet Info

Destination: 10.180.202.63
Protocol: TCP
Length: 362
Info: Ether / IP / TCP 180.76.151.202:htp > 10.180.202.63:60222 PA / Raw
Details: Ethernet Header:
Destination MAC: e8:fb:1c:1b:97:b0
Source MAC: 54:c6:ff:7b:58:02
Type: 0x800

Payload

UTF-8

HTTP/1.1 302 Found
Server: BLB/23.08.0.7
Date: Tue, 17 Dec 2024 14:30:09 GMT
Content-Length: 0
Connection: keep-alive
Location: http://180.76.151.202/portal/course/7182/26743.mooc
Content-Language: zh-CN
Expires: Tue, 17 Dec 2024 14:30:09 GMT
Cache-Control: max-age=0
X-Frame-Options: SAMEORIGIN

图 8: cnmooc 响应报文

[illegible]

图 9: 百度响应报文

3.2.2 报文分析测试

选取任意一个报文查看概要分析，该报文格式为 Ether/IP/TCP/HTTPS

```
Packet No: 159
Time: 2024-12-17 22:37:27
Source: 10.180.202.63
Destination: 110.242.69.176
Protocol: TCP
Length: 571
Info: Ether / IP / TCP 10.180.202.63:60923 > 110.242.69.176:https PA / Raw
Details: Ethernet Header:
    Destination MAC: 54:c6:ff:7b:58:02
    Source MAC: e8:fb:1c:1b:97:b0
    Type: 0x800
```

图 10: 报文分析概要部分

概要分析以下是各层的详细分析结果


```

IP Header:
Version: 4 (IPv4)
Header Length: 20 bytes
Total Length: 783 bytes
Identification: 0x17c0
Flags: DF
TTL: 43
Protocol: 6
Source IP: 110.242.69.176
Destination IP: 10.180.202.63

TCP Header:
Source Port: 443
Destination Port: 60923
Sequence Number: 1312839327
Acknowledgment Number: 671800516
Data Offset: 20 bytes
Flags: PA
Window Size: 2592
Checksum: 0xc3c9
Urgent Pointer: 0
Options: []

HTTPS Info:
[] [] [] [] L p:fx[]& EA!P[]`hIY[]s/:n2c7T[],b7Ez[]K[]|x\Eh-+ort04Ma
/[] []uk[]</z[] [8/9Q2sR9T+[] [] R2$[]6[]F[]R[]gp=I[] [] [0?r!ldFM tUw[]h[] (54, [] []iA[]g$ZC. |X([]
    
```

图 11: 报文详细分析部分

图 12: ASCII

3.3 报文重组测试

使用附录 B 中的代码发送分片报文，并进行测试。在列表中查看其中一个分片报文，很明显不是完整的。点击报文重组，查看重组后内容，得到完整报文 payload。

No	Time	Source	Destination	Protocol	Length	Info
1	2024-12-17 23:26:13	10.180.202.63	192.168.1.1	UDP	1034	Ether / IP / UDP / DNS Qry b:'
2	2024-12-17 23:26:13	10.180.202.63	192.168.1.1	UDP	1034	Ether / 10.180.202.63 > 192.168.1.1 udp frag:125 / Raw
3	2024-12-17 23:26:13	10.180.202.63	192.168.1.1	UDP	1034	Ether / 10.180.202.63 > 192.168.1.1 udp frag:250 / Raw
4	2024-12-17 23:26:13	10.180.202.63	192.168.1.1	UDP	957	Ether / 10.180.202.63 > 192.168.1.1 udp frag:375 / Raw
5	2024-12-17 23:26:31	10.180.202.63	119.29.29.29	UDP	92	Ether / IP / UDP / DNS Qry b'mobile.events.data.microsoft.com.'
6	2024-12-17 23:26:31	10.180.202.63	223.5.5.5	UDP	92	Ether / IP / UDP / DNS Qry b'mobile.events.data.microsoft.com.'
7	2024-12-17 23:26:31	223.5.5.5	10.180.202.63	UDP	212	Ether / IP / UDP / DNS Ans b'mobile.events.data.trafficmanager.net.'
8	2024-12-17 23:26:31	119.29.29.29	10.180.202.63	UDP	213	Ether / IP / UDP / DNS Ans b'mobile.events.data.trafficmanager.net.'
9	2024-12-17 23:26:31	2403:d400:1000:12:89b8	2400:da00:6666	UDP	104	Ether / IPv6 / UDP / DNS Qry b'tpstelemetry.tencent.com.'
10	2024-12-17 23:26:31	10.180.202.63	114.114.114.114	UDP	84	Ether / IP / UDP / DNS Qry b'tpstelemetry.tencent.com.'
11	2024-12-17 23:26:31	114.114.114.114	10.180.202.63	UDP	112	Ether / IP / UDP / DNS Ans 240e:97c2f2:5c
12	2024-12-17 23:26:31	10.180.202.63	119.29.29.29	UDP	101	Ether / IP / UDP / DNS Qry b'westus-0.in.applicationinsights.azure.com
13	2024-12-17 23:26:31	10.180.202.63	223.5.5.5	UDP	101	Ether / IP / UDP / DNS Qry b'westus-0.in.applicationinsights.azure.com

Packet Info: Destination IP: 192.168.1.1

Unknown Payload: or cursus elit, eget dictum orci felis non nisi. Praesent bibendum enim vel sapien ultrices, at lacinia felis dapibus. Vestibulum sit amet eros sed sapien consectetur pretium.</p>
 <h3>Features</h3>

 Feature 1: Detailed description about feature 1.
 Feature 2: Detailed description about feature 2.
 Feature 3: Detailed description about feature 3.
 Feature 4: Detailed description about feature 4.
 Feature 5: Detailed description about feature 5.

 <h2>Data Table Example</h2>
 <table>
 <thead>
 <tr>
 <th>ID</th>
 <th>Name</th>
 <th>Description</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <td>1</td>
 <td>Feature 1</td>
 <td>Detailed description about feature 1.</td>
 </tr>
 <tr>
 <td>2</td>
 <td>Feature 2</td>
 <td>Detailed description about feature 2.</td>
 </tr>
 <tr>
 <td>3</td>
 <td>Feature 3</td>
 <td>Detailed description about feature 3.</td>
 </tr>
 <tr>
 <td>4</td>
 <td>Feature 4</td>
 <td>Detailed description about feature 4.</td>
 </tr>
 <tr>
 <td>5</td>
 <td>Feature 5</td>
 <td>Detailed description about feature 5.</td>
 </tr>
 </tbody>
 </table>
 </h2>Data Table Example</h2>
 </p>
 </div>
 <div data-bbox="404 888 590 905" data-label="Caption">
 <p>图 13: 报文分片之一</p>
 </div>
 <div data-bbox="489 936 506 952" data-label="Page-Footer">
 <p>9</p>
 </div>
 </div>

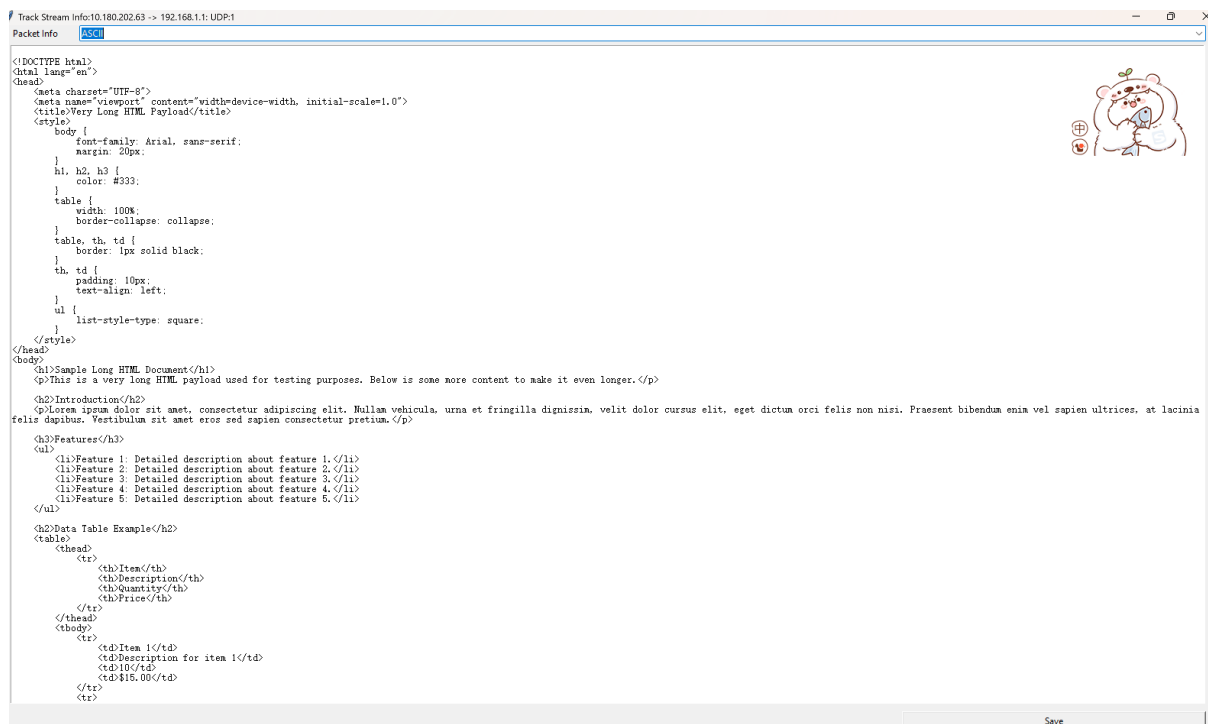


图 14: 重组报文

3.4 筛选功能

在筛选器中选择筛选协议 tcp, 并保存。随后开始侦听, 可以看到侦听到的都是 TCP 包。

No	Time	Source	Destination	Protocol	Length	Info
1	2024-12-17 23:39:22	120.233.70.120	10.180.202.63	TCP	60	Ether / IP / TCP 120.233.70.120:11001 > 10.180.202.63:50709 A / Padding
2	2024-12-17 23:39:22	10.180.202.63	120.233.70.120	TCP	54	Ether / IP / TCP 10.180.202.63:50709 > 120.233.70.120:11001 A
3	2024-12-17 23:39:23	10.180.202.63	120.233.70.120	TCP	55	Ether / IP / TCP 10.180.202.63:49982 > 120.233.70.120:11001 A / Raw
4	2024-12-17 23:39:23	120.233.70.120	10.180.202.63	TCP	66	Ether / IP / TCP 120.233.70.120:11001 > 10.180.202.63:49982 A
5	2024-12-17 23:39:23	10.180.202.63	58.205.220.16	TCP	55	Ether / IP / TCP 10.180.202.63:62313 > 58.205.220.16:https A / Raw
6	2024-12-17 23:39:23	58.205.220.16	10.180.202.63	TCP	66	Ether / IP / TCP 58.205.220.16:https > 10.180.202.63:62313 A
7	2024-12-17 23:39:23	10.180.202.63	13.107.21.239	TCP	55	Ether / IP / TCP 10.180.202.63:50674 > 13.107.21.239:https A / Raw
8	2024-12-17 23:39:23	13.107.21.239	10.180.202.63	TCP	66	Ether / IP / TCP 13.107.21.239:https > 10.180.202.63:50674 A
9	2024-12-17 23:39:23	10.180.202.63	222.192.186.38	TCP	55	Ether / IP / TCP 10.180.202.63:49784 > 222.192.186.38:https A / Raw
10	2024-12-17 23:39:24	222.192.186.38	10.180.202.63	TCP	66	Ether / IP / TCP 222.192.186.38:https > 10.180.202.63:49784 A
11	2024-12-17 23:39:24	120.233.171.103	10.180.202.63	TCP	115	Ether / IP / TCP 120.233.171.103:11001 > 10.180.202.63:63920 PA / Raw
12	2024-12-17 23:39:24	10.180.202.63	120.233.171.103	TCP	111	Ether / IP / TCP 10.180.202.63:63920 > 120.233.171.103:11001 PA / Raw
13	2024-12-17 23:39:24	10.180.202.63	120.233.171.103	TCP	111	Ether / IP / TCP 10.180.202.63:63920 > 120.233.171.103:11001 PA / Raw

图 15: 筛选 TCP 报文

3.5 保存功能

首先, 重组后的报文内容可以保存, 以 3.2.3 中的报文为例: 将其保存为 output1, 打开即可看到报文内容。其次, 侦听记录可以保存成 csv (点击主界面菜单栏 File - Save)

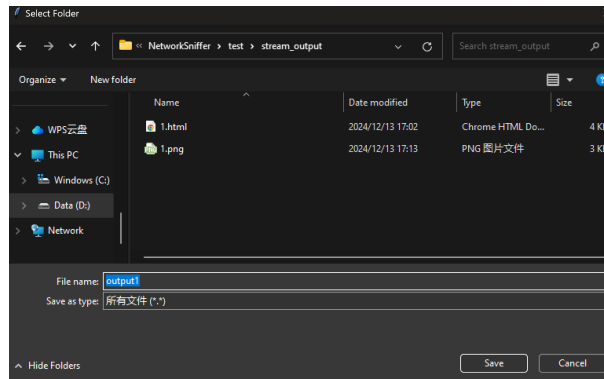


图 16: 保存界面



图 17: 保存文件

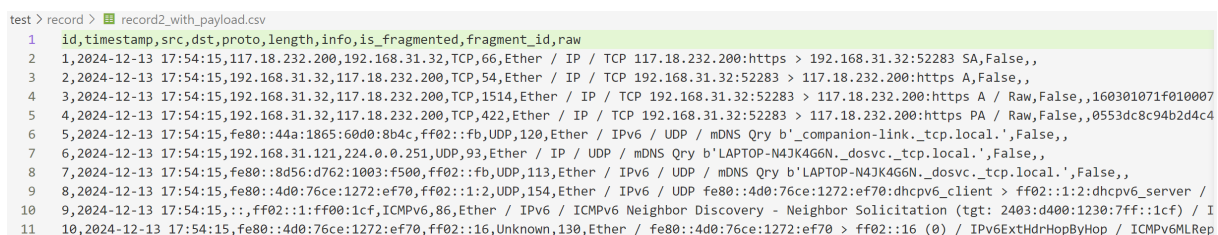


图 18: 侦听结果保存

4 写在最后

4.1 发布地址

- Github(大概率设 private 了): https://github.com/sensuschung/network_sniffer

4.1.1 遇到的问题与解决办法

最主要的问题是，在接收报文过多（比如开着腾讯会议且有音频传输）的时候，由于处理速度慢于接收速度，会导致程序卡顿，有可能已经点击停止，但是还在继续处理报文和更新 UI。

后续为了缓解该问题，采取了几个措施

- 将程序分为接收线程和处理线程，并增加增加接收队列和处理队列，使得二者可

以异步进行，并且将具体报文的分析延后到点击该报文时处理

- 减少 UI 更新频率，每十个 Packet/程序停止侦听时更新 UI

但是该问题还是存在，可能需要引入多线程池之类、并且采取效率更高的 UI。

4.1.2 体会与建议

- 有时候写代码的时候，还是更多的需要人力干预，大模型并不能解决很多较为复杂的问题，又或者说，大模型写出来的代码有时候会有些冗余
- 还有这学期因为排课原因，进度确实有点快

A 与大模型交互过程

在项目中使用的大模型主要为 **github copilot**。其部分交互过程如下：

1. 代码生成：基本内容的构建

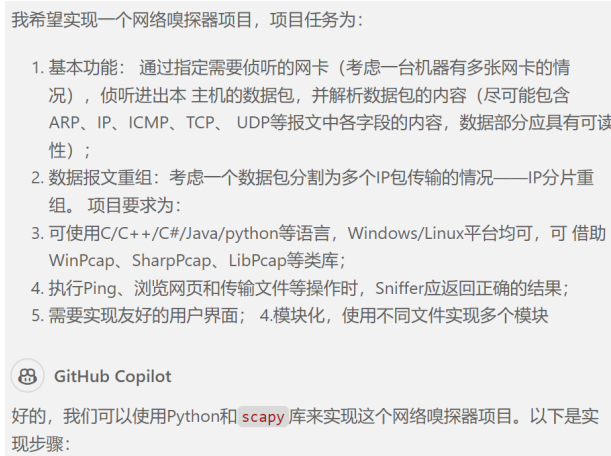


图 19: 描述基本任务

2. 功能补全示例：网卡选择功能

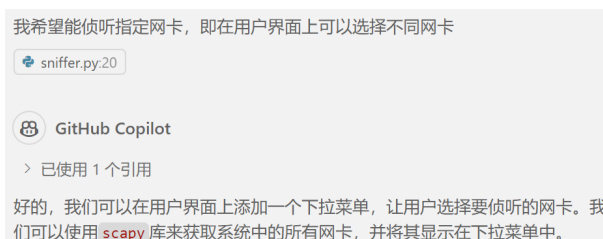


图 20: 描述功能 1: 网卡选择



图 21: debug: 传递正确的接口名称

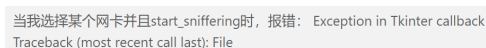


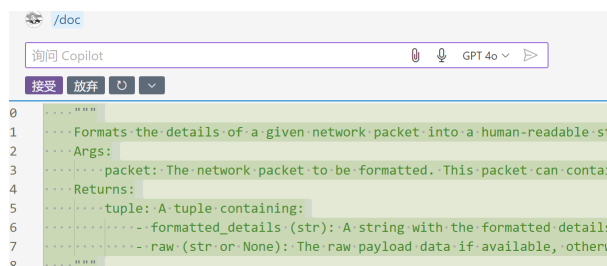
图 22: debug: 网卡选择后监听报错

3. debug 示例

```
安装psutil时报错 Downloading
https://pypi.tuna.tsinghua.edu.cn/packages/26/10/2a30b13c61e7cf937f4adf907
10776b7918ed0a9c434e2c38224732af310/psutil-6.1.0.tar.gz (508kB) |
512kB 2.2MB/s Installing
build dependencies ... done Getting requirements to build wheel ... done
Preparing wheel metadata ... done Building wheels for collected packages:
psutil Building wheel for psutil (PEP 517) ... error ERROR: Command errored out
```

图 23: 安装报错:psutil 需要编译 C 扩展

4. 生成文档



```
0
1 ...Formats the details of a given network packet into a human-readable st
2 ...Args:
3 ...packet: The network packet to be formatted. This packet can contai
4 ...Returns:
5 ...tuple: A tuple containing:
6 ...formatted_details (str): A string with the formatted details
7 ...raw (str or None): The raw payload data if available, otherw
8
```

图 24: 为 format_packet 函数生成文档

B 报文重组测试代码

```
1 from scapy.all import IP, UDP, TCP, Raw, fragment, send, conf
2
3 conf.iface = "*****"
4 # conf.iface = "*****"
5
6 html_payload = """
7 <!DOCTYPE html>
8 <html lang="en">
9 <head>
10     <meta charset="UTF-8">
11     <meta name="viewport" content="width=device-width,
12         initial-scale=1.0">
13     <title>Very Long HTML Payload</title>
14     <style>
15         body {
16             font-family: Arial, sans-serif;
```

```

16         margin: 20px;
17     }
18     h1, h2, h3 {
19         color: #333;
20     }
21     table {
22         width: 100%;
23         border-collapse: collapse;
24     }
25     table, th, td {
26         border: 1px solid black;
27     }
28     th, td {
29         padding: 10px;
30         text-align: left;
31     }
32     ul {
33         list-style-type: square;
34     }
35 </style>
36 </head>
37 <body>
38     <h1>Sample Long HTML Document</h1>
39     <p>This is a very long HTML payload used for testing
40         purposes. Below is some more content to make it even
41         longer.</p>
42
43     <h2>Introduction</h2>
44     <p>Lorem ipsum dolor sit amet, consectetur adipiscing
45         elit. Nullam vehicula, urna et fringilla dignissim,
46         velit dolor cursus elit, eget dictum orci felis non
47         nisi. Praesent bibendum enim vel sapien ultrices, at
48         lacinia felis dapibus. Vestibulum sit amet eros sed
49         sapien consectetur pretium.</p>
50
51     <h3>Features</h3>
52     <ul>

```

```

46         <li>Feature 1: Detailed description about feature
47             1.</li>
48         <li>Feature 2: Detailed description about feature
49             2.</li>
50         <li>Feature 3: Detailed description about feature
51             3.</li>
52         <li>Feature 4: Detailed description about feature
53             4.</li>
54         <li>Feature 5: Detailed description about feature
55             5.</li>
56     </ul>
57
58     <h2>Data Table Example</h2>
59     <table>
60         <thead>
61             <tr>
62                 <th>Item</th>
63                 <th>Description</th>
64                 <th>Quantity</th>
65                 <th>Price</th>
66             </tr>
67         </thead>
68         <tbody>
69             <tr>
70                 <td>Item 1</td>
71                 <td>Description for item 1</td>
72                 <td>10</td>
73                 <td>$15.00</td>
74             </tr>
75             <tr>
76                 <td>Item 2</td>
77                 <td>Description for item 2</td>
78                 <td>5</td>
79                 <td>$25.00</td>
80             </tr>
81             <tr>
82                 <td>Item 3</td>
83                 <td>Description for item 3</td>
84                 <td>3</td>
85                 <td>$8.00</td>
86             </tr>
87         </tbody>
88     </table>
89
90     <h2>Footer</h2>
91     <p>Page 1 of 1</p>
92 
```



```

78         <td>Description for item 3</td>
79         <td>2</td>
80         <td>$100.00</td>
81     </tr>
82     <tr>
83         <td>Item 4</td>
84         <td>Description for item 4</td>
85         <td>20</td>
86         <td>$7.50</td>
87     </tr>
88 </tbody>
89 </table>
90
91 <h2>Additional Information</h2>
92 <p>Lorem ipsum dolor sit amet, consectetur adipiscing
    elit. Vestibulum congue ex non urna facilisis, at
    scelerisque magna laoreet. Proin eget dolor vitae
    nisl vehicula sollicitudin ac a elit. Donec eget
    felis magna. Suspendisse potenti. Nunc malesuada
    libero sit amet libero sollicitudin, quis efficitur
    erat vestibulum.</p>
93 <p>Phasellus malesuada, nunc eget varius aliquam,
    ligula lectus viverra est, id fermentum risus felis
    vitae dui. Donec faucibus lorem ut ex fermentum, nec
    vulputate libero laoreet. Maecenas ultrices diam
    sed leo vulputate, in dictum neque pellentesque. Nam
    euismod, nisi in facilisis rutrum, sem lectus
    tincidunt mauris, a tristique ex nisi id lectus.</p>
94 <p>Pellentesque habitant morbi tristique senectus et
    netus et malesuada fames ac turpis egestas. Etiam
    fringilla, felis id ultrices vehicula, urna erat
    gravida risus, et lobortis enim lorem et risus.</p>
95
96 <h2>Conclusion</h2>
97 <p>In conclusion, this is an example of a very long
    HTML document used to demonstrate payload handling.
    The content is not meaningful but simply serves as a

```

```
test for length and structure.</p>
98
99 <h2>Final Remarks</h2>
100 <p>Thank you for reviewing this document. It is
    intentionally lengthy to provide a substantial
    payload for testing network transmission of large
    data packets.</p>
101 </body>
102 </html>
103 """.encode()
104
105 a_payload = b"A"*5000
106
107 image_path = "./test/src_file/start.png"
108 with open(image_path, "rb") as img_file:
109     image_data = img_file.read()
110
111 pkt_tcp = IP(dst="192.168.1.1") / TCP(dport=12345, sport
    =54321, flags="S") / html_payload # 超过MTU, 会自动进
    行分片
112 pkt_udp = IP(dst="192.168.1.1") / UDP(dport=12345) /
    html_payload
113 # print(pkt)
114
115 print(len(Raw(load=image_data)))
116
117 fragments = fragment(pkt_tcp, fragsize=1000)
118
119 send(fragments)
120
121 fragments_udp = fragment(pkt_udp, fragsize=1000)
122 send(fragments_udp)
```